

Licenciatura en Sistemas de Información Desarrollo de aplicaciones Móviles - 2026

01 Network QoS Monitor

Analizador y visualizador de calidad de red móvil en tiempo real con mapeo de cobertura personal

Área temática: Redes de Datos, Telecomunicaciones y Telecomunicaciones Satelitales

Tecnología: React Native

Nivel: Medio-Avanzado

Licenciatura en Sistemas de Información

Desarrollo de aplicaciones Móviles - 2026

01. Introducción y contexto

Este trabajo práctico propone el desarrollo de una aplicación móvil multiplataforma (Android/iOS) construida con React Native que mida, almacene y visualice en tiempo real la calidad de la conexión de red del dispositivo. La aplicación debe correlacionar las métricas de red con la ubicación geográfica del usuario, generando un historial navegable y un mapa de cobertura personal basado en mediciones propias.

El proyecto obliga a trabajar por debajo de las abstracciones habituales del ecosistema React Native: acceso a información de telefonía celular mediante módulos nativos, medición de latencia real mediante sockets, y procesamiento de datos en segundo plano. Es representativo de los problemas reales que enfrenta un equipo de ingeniería de una operadora de telecomunicaciones al monitorear la experiencia de red del usuario final (QoE/QoS).

02. Objetivos

02.1 Objetivo general

Diseñar e implementar una aplicación móvil capaz de medir parámetros de calidad de servicio (QoS) de la red de datos del dispositivo, persistir el historial de mediciones y representarlo geográficamente y temporalmente.

02.2 Objetivos específicos

- Determinar el tipo de conexión activa (WiFi, 3G, 4G/LTE, 5G NR) y sus parámetros básicos.
- Medir latencia (RTT), jitter y pérdida de paquetes mediante sondas propias sobre sockets TCP/UDP.
- Implementar un test de throughput (descarga/subida) contra un backend de referencia.
- Georreferenciar cada medición y construir un mapa de calor (heatmap) de cobertura.
- Persistir el historial localmente y permitir su exportación.
- Ejecutar muestreos periódicos en segundo plano, incluso con la app cerrada o el dispositivo bloqueado.

Licenciatura en Sistemas de Información

Desarrollo de aplicaciones Móviles - 2026

03. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	Detectar y mostrar el tipo de red activa y el operador (cuando esté disponible)	Alta
RF-02	Medir RTT (ping) contra al menos 3 hosts configurables, mostrando min/avg/max/jitter	Alta
RF-03	Ejecutar un test de descarga y subida y calcular throughput en Mbps	Alta
RF-04	Registrar cada medición con timestamp y coordenadas GPS	Alta
RF-05	Visualizar el historial en un mapa con heatmap de intensidad de señal/calidad	Alta
RF-06	Graficar series temporales de latencia y throughput por sesión	Media
RF-07	Ejecutar mediciones periódicas en background y notificar degradaciones severas	Media
RF-08	Exportar el historial de mediciones a CSV/JSON	Baja
RF-09	Filtrar el historial por tipo de red, rango de fechas y zona geográfica	Baja

04. Requisitos técnicos y stack sugerido

04.1 Frontend / aplicación móvil

- React Native (CLI, no Expo Go puro — se requieren módulos nativos custom).
- @react-native-community/netinfo — detección de tipo de conexión y estado.
- react-native-tcp-socket — sondas de latencia RTT reales sobre TCP.
- Módulo nativo propio (Kotlin/Swift) para acceder a TelephonyManager (Android) / CoreTelephony (iOS) y exponer RSSI, tipo de red celular y operador vía Native Modules / TurboModules.
- react-native-background-fetch o Headless JS (Android) para muestreo periódico en segundo plano.
- react-native-maps — visualización de heatmap (Heatmap overlay) y marcadores de medición.
- victory-native o react-native-svg-charts — series temporales de latencia/throughput.
- WatermelonDB o SQLite (react-native-sqlite-storage) — persistencia local performante.
- @notifee/react-native — notificaciones locales ante degradación de calidad.

Licenciatura en Sistemas de Información

Desarrollo de aplicaciones Móviles - 2026

04.2 Backend de referencia (requerido para el test de throughput)

- Servicio mínimo (Node.js/Express o Fastify) con endpoints de descarga de payload de tamaño fijo y de subida (echo), desplegable en un contenedor simple.
- Medición de tiempo transcurrido en cliente para calcular Mbps con corrección por tamaño de payload.

05. Arquitectura propuesta

Se sugiere una arquitectura en capas que separe claramente la adquisición de datos nativos, el motor de medición, la persistencia y la capa de presentación:

Capa	Responsabilidad	Tecnología
Native Bridge	Exponer RSSI, tipo de red celular, operador	Kotlin / Swift + TurboModules
Measurement Engine	Orquestar pings, throughput test y muestreo periódico	TypeScript, sockets, background tasks
Persistence Layer	Guardar mediciones, exponer queries por fecha/zona	WatermelonDB / SQLite
Geo Layer	Obtener y cachear ubicación, calcular heatmap	react-native-geolocalization-service
Presentation Layer	Mapa, gráficos, listado histórico	react-native-maps, victory-native
Sync/Export (opcional)	Backend propio para test de throughput y exportación remota	Node.js / Express

Un punto de diseño central es evitar bloquear el hilo de JavaScript durante las mediciones: los sockets y timers de las sondas deben correr en un contexto que no degrade la UI, y los resultados deben comunicarse a la capa de presentación mediante un store reactivo (Zustand, Redux Toolkit o Context + useReducer).

06. Etapas de desarrollo sugeridas

Etapas	Contenido	Duración estimada
1	Setup del proyecto, permisos (ubicación, teléfono), integración de NetInfo	1 semana
2	Módulo nativo de telefonía (RSSI, operador, tipo de red celular)	1,5 semanas
3	Motor de ping/jitter sobre TCP sockets y test de throughput	1,5 semanas

Licenciatura en Sistemas de Información

Desarrollo de aplicaciones Móviles - 2026

Etapa	Contenido	Duración estimada
4	Persistencia local y georreferenciación de mediciones	1 semana
5	Visualización: mapa con heatmap y gráficos de series temporales	1,5 semanas
6	Background fetch, notificaciones y exportación de datos	1 semana
7	Testing, pulido de UX y documentación final	1 semana

07. Entregables

- Código fuente completo en repositorio Git con historial de commits significativo.
- APK/IPA de prueba o build accesible vía Expo Dev Client / TestFlight interno.
- Documento técnico (README extendido) describiendo arquitectura, decisiones de diseño y limitaciones conocidas.
- Backend de referencia para el test de throughput, con instrucciones de despliegue.
- Video demo (3-5 min) mostrando: detección de red, medición de latencia/throughput, mapa de calor con al menos 2 sesiones de datos reales.

08. Recursos y bibliografía sugerida

- Documentación oficial de React Native — Native Modules / TurboModules.
- Android: android.telephony.TelephonyManager — documentación oficial de Android Developers.
- iOS: CoreTelephony framework — Apple Developer Documentation.
- RFC 2544 — Benchmarking Methodology for Network Interconnect Devices (referencia conceptual para metodología de medición).
- Documentación de OpenCellID y Mozilla Location Service (para eventual validación cruzada de cobertura).